

《编程农场》阶段体验周期与数值 / 内容填充设计拆解

一、核心体验概括

《编程农场》的核心不是“种田”，而是把种田作为一个可视化的编程反馈系统。

玩家一开始面对的是一个很小的农场、一个无人机、一组简单 API：

```
move()
plant()
harvest()
can_harvest()
get_world_size()
```

看起来像是普通自动化游戏，但真正的体验循环是：

```
发现重复劳动
→ 写脚本自动化
→ 产出资源
→ 解锁新机制 / 新 API
→ 旧脚本失效或效率不足
→ 重构脚本
→ 进入更高层次的自动化
```

因此，这款游戏的内容填充方式并不是单纯增加作物数量，而是不断引入新的“编程问题”：

```
草：循环与遍历
树：空间布局
南瓜：全局状态检测
肥料 / 奇怪物质：资源转换链
迷宫：寻路
向日葵：测量与最值选择
仙人掌：排序
混合种植：局部依赖与图关系
巨大农场：并发 / 多无人机调度
恐龙：路径规划与自碰撞约束
排行榜：完整自动化与效率压榨
```

这使得游戏的成长曲线不是“数值越来越大”那么简单，而是“玩家代码能力越来越复杂”。

二、阶段一：手动劳动到基础脚本

体验目标

让玩家理解最基础的自动化价值。

玩家最初要做的事非常朴素：

```
移动  
种植  
等待  
收获  
重复
```

很快玩家会发现手动操作低效，于是自然产生写循环的需求。

主要资源

```
Hay  
Wood  
Water  
Carrot
```

主要机制

这个阶段的核心 API 通常是：

```
move()  
plant()  
harvest()  
can_harvest()  
get_entity_type()  
get_ground_type()  
till()
```

玩家第一次遇到的设计问题是：

```
如何遍历整张地图？
```

最直接的结构是：

```
for x in world:  
    for y in world:  
        处理当前格子
```

```
move(North)
move(East)
```

数值设计思路

早期资源阈值不需要很复杂。资源需求的作用不是构建深度经济，而是迫使玩家建立第一个自动化循环。

这个阶段的数值应该具备三个特点：

1. 单次收获反馈快
2. 升级成本足够低，能快速验证脚本收益
3. 资源瓶颈清晰，玩家知道下一步该补什么

例如：

Hay 不够 → 种草
Wood 不够 → 种树 / 灌木
Carrot 不够 → 学会耕地和种胡萝卜

内容填充设计

此阶段的内容不是靠“更多作物”填充，而是靠“更多基础动作”填充：

地块类型
作物类型
是否成熟
是否需要耕地
是否需要水

这些看似小机制，会迫使玩家写出第一个真正的“农场状态机”。

三、阶段二：阈值轮作与基础资源链

体验目标

从“能自动跑”变成“能稳定生产”。

玩家开始写类似这样的资源策略：

Hay 不足 → 种草
Wood 不足 → 种树
Carrot 不足 → 种胡萝卜
Pumpkin 不足 → 种南瓜

这时游戏的体验重点变成：

生产链不能断

例如南瓜本身是高级作物，但南瓜种植会消耗胡萝卜，因此胡萝卜变成了南瓜生产线的“弹药”。

主要资源关系

草 / 木头：基础升级材料
胡萝卜：南瓜前置资源
南瓜：肥料、升级、中期资源链核心
水：加速资源，但会被快速消耗

关键设计点：显性资源与隐性瓶颈

这个阶段有一个很重要的设计思路：

高级作物不是凭空高级，它会反过来消耗低级资源。

例如：

南瓜收益高
但南瓜依赖胡萝卜
胡萝卜依赖基础生产

于是玩家不能只写：

一直种最高级作物

而必须写：

高级作物消耗低级资源
低级资源下降后自动回补

这就是游戏第一次逼玩家理解“生产链”。

数值设计思路

这个阶段的资源阈值如果只使用单一线，会出现抖动：

胡萝卜刚低于阈值 → 切胡萝卜
胡萝卜刚高于阈值 → 切南瓜
南瓜消耗胡萝卜 → 又切胡萝卜

更好的数值策略是“滞回区间”：

胡萝卜低于低线 → 进入胡萝卜恢复模式
胡萝卜高于高线 → 退出胡萝卜恢复模式

这实际上是在把简单阈值系统升级成轻量状态机。

内容填充设计

此阶段的新内容最好围绕“同一套 API 产生不同策略”展开。

例如：

树不能密集种，需要棋盘格
南瓜不能成熟一个收一个，要整片成熟再收
水不能无脑用，要对高价值作物使用

这些内容不需要引入复杂语法，但能显著提升策略深度。

四、阶段三：南瓜、肥料与奇怪物质

体验目标

把普通农场推进到“资源转换链”。

南瓜之后，玩家开始接触：

Fertilizer
Weird Substance
Maze
Gold

这是一条很典型的中期资源链：

南瓜 / 资源 → trade 肥料
肥料感染植物
收获获得 Weird Substance
Weird Substance 用于生成迷宫
迷宫产出 Gold

机制设计价值

这条链非常关键，因为它把游戏从“农作物生产”推向“系统性转换”。

玩家不再只是问：

我该种什么？

而是开始问：

我缺 Gold
Gold 来自 Maze
Maze 需要 Weird Substance
Weird Substance 需要 Fertilizer
Fertilizer 需要 trade
trade 又需要基础资源

于是玩家代码开始从“轮作脚本”升级为“经济调度器”。

数值设计思路

这里的设计重点是：

不要让玩家只堆一种资源。

如果 Gold 很重要，但 Gold 需要 Weird Substance，而 Weird Substance 又需要 Fertilizer，那么系统天然会产生多层瓶颈。

这是一种很好的中期数值填充方式：

表层目标：Gold
中间材料：Weird Substance
消耗品：Fertilizer
基础供给：Pumpkin / Hay / Wood / Carrot

玩家为了获得 Gold，不得不维护整条链。

设计风险

这个阶段也容易出现“策略锁死”：

下一个升级缺 Gold
→ 一直刷 Gold
→ 其他生产链断粮

因此自动化系统必须有优先级：

生产链安全 > 升级缺口 > 软储备 > 兜底生产

这是游戏中期代码设计的分水岭。

五、阶段四：迷宫与 Gold 经济

体验目标

引入“寻路”问题。

迷宫的核心作用不是“探索内容”，而是把 Weird Substance 转换成 Gold。
因此从生产效率角度看，迷宫本质上是：

Gold 转换器

玩家第一次会写简单解法：

左手法
右手法
贴墙走

迷宫尺寸的数值问题

迷宫不是越大越好。

大迷宫意味着：

单次宝箱收益可能更高
但 Weird Substance 成本更高
寻路时间更长
卡住风险更大
单位时间 Gold 不一定更高

在简单贴墙法阶段，小迷宫往往更高效：

小迷宫
→ 快速生成
→ 快速找到宝箱
→ 快速收 Gold

等玩家解锁更强 API、能记录路径、能复用迷宫后，大迷宫才更有意义。

内容填充设计

迷宫阶段的内容填充方式很聪明，因为它引入了完全不同于种田的编程问题：

```
移动受阻  
路径选择  
方向状态  
循环退出条件  
失败保护
```

这让玩家从“遍历网格”进入“路径算法”。

即便最简单的迷宫也会迫使玩家写：

```
turn_left()  
turn_right()  
can_move()  
step_limit
```

这就是内容层面的横向展开。

六、阶段五：字典、get_cost、unlock 与自动经济系统

体验目标

让玩家从“固定脚本”进入“自适应脚本”。

当玩家解锁：

```
Dictionaries  
get_cost()  
unlock()  
import
```

以后，游戏真正进入自动化中期。

之前的脚本是：

```
我手动决定下一步升级什么  
脚本只负责刷资源
```

现在变成：

脚本读取升级成本
判断缺什么
自动生产对应资源
资源够了自动 unlock

设计意义

这是非常重要的体验跃迁。
因为它不是简单给玩家一个新资源，而是给玩家一个“控制系统自身”的能力。

玩家的代码开始具备：

目标检测
成本评估
资源补缺
自动购买
动态调度

这已经接近一个小型经济 AI。

数值设计思路

这个阶段的数值填充不应该继续依赖固定阈值，例如：

Hay 12000
Wood 10000
Carrot 6000

因为地图变大、升级成本变高、无人机数量变化后，固定数值会频繁失效。

更好的策略是动态化：

基础储备 = 地块数量 × 每格储备
高级储备 = 地块数量 × 每格储备
升级目标 = get_cost(下一个升级)

于是资源目标不再是固定数字，而是跟随：

地图面积
无人机数量
下一个升级成本
当前科技树进度

这就是数值系统从“手工阈值”进入“动态目标”的关键。

七、阶段六：向日葵与 Power

体验目标

引入“测量”和“最值选择”。

向日葵不是普通作物。

正确策略不是成熟就收，而是：

扫描所有成熟向日葵
measure() 获取花瓣数
找到花瓣数最大的那朵
收割最大值
获得 Power

这里引入的编程问题是：

遍历
测量
记录最大值
记录坐标
移动到目标
收割

这是一个经典的“find max”问题。

Power 的特殊数值属性

Power 和 Gold、Pumpkin 不一样。

它不是稳定库存，而是会随着无人机移动持续消耗的“流动燃料”。

因此 Power 不能用普通库存策略：

Power < 目标 → 一直刷 Power

这样会出现循环：

刷向日葵
→ 移动消耗 Power
→ 仍然低于目标
→ 继续刷向日葵

正确策略应该是：

只有升级明确需要 Power 时，节流式刷 Power
收一朵最大花瓣
立刻尝试 unlock
然后退出 Power 模式

内容填充设计

向日葵阶段的设计非常优雅，因为它把“作物差异”转化成“算法差异”。

普通作物是：

成熟 → 收

南瓜是：

整片成熟 → 收

向日葵是：

找最大 → 收

同样是收获，但背后对应完全不同的代码结构。

八、阶段七：巨大农场与多无人机

体验目标

从单线程自动化进入并行调度。

当玩家解锁巨大农场后，生产逻辑从：

一个无人机扫完整张图

变成：

多个无人机按列并行处理

这时内容的核心不再是作物，而是“调度”。

玩家需要处理：

```
spawn_drone()  
wait_for()  
任务分配  
列任务  
返回结果合并  
不同无人机避免互相干扰
```

数值设计影响

多无人机会显著改变资源系统。

同一张地图，在单无人机时期可能需要：

```
先补草  
再补胡萝卜  
再补南瓜
```

到了多无人机阶段，生产速度变快，但消耗也更快：

```
移动消耗 Power 更快  
水消耗更快  
种子链消耗更快
```

因此数值目标必须跟随：

```
地块数  
无人机数  
移动消耗  
生产速度
```

内容填充设计

巨大农场本质上不是单纯增加效率，而是打开了新的代码架构需求：

```
单文件脚本不够用  
需要模块化  
需要任务系统  
需要共享状态  
需要结果汇总
```

所以它自然推动玩家使用：

```
import  
多文件
```

```
farm_mode
farm_unlock
farm_drone
farm_field
```

游戏通过数值压力迫使玩家走向代码架构化。

九、阶段八：仙人掌、混合种植与高级空间问题

仙人掌

仙人掌真正的玩法不是普通种植，而是排序。

它会引入：

```
measure()
swap()
比较相邻地块
二维排序
错误顺序成就
```

这不是当前主生产链的一部分，而是后期算法挑战。

从收益最大化角度看，仙人掌不适合太早深度接入主循环，因为它会显著增加逻辑复杂度。

混合种植

混合种植引入的是“局部依赖”：

```
当前作物希望某个坐标种某个 companion
```

这会让玩家开始面对：

```
局部最优
坐标跳转
伴生关系
路径额外成本
收益是否抵消移动损耗
```

它不是简单的产量加成，而是一个图关系系统。

设计思路

这两个系统都属于“高级内容填充”。

它们的共同点是：

不只是提高产量
而是引入新的代码结构

仙人掌要求排序，混合种植要求依赖管理。
这比单纯增加第十种、第十一种作物更有设计价值。

十、阶段九：恐龙与完全不同的移动规则

体验目标

引入另一套游戏规则。

恐龙帽会把普通移动系统变成类似“贪吃蛇”的系统：

无人机移动
尾巴跟随
尾巴占据格子
撞到尾巴则移动失败
长度越长，骨头结算越高

这意味着恐龙不能简单混进普通农场主循环。

普通农场依赖：

go_to()
按列扫描
多无人机并行

而恐龙要求：

安全路径
避免自撞
规划路线
填满农场
独立模式

数值设计思路

恐龙的收益通常来自长度平方级结算，因此它适合后期：

农场足够大
移动速度足够高

主资源链稳定
可以暂停普通生产
专门跑恐龙脚本

它不是中期收益最大化路线，而是后期独立分支。

内容填充设计

恐龙是非常好的“后期横向玩法”，因为它彻底改变了玩家已有代码假设。这让游戏不会只停留在“更大农场、更高数值”，而是引入新的算法空间。

十一、阶段十：排行榜与完全自动化

体验目标

从“能长期运行”变成“从零到目标的最优流程”。

排行榜不是普通内容，而是要求玩家完成一整套自动化：

开局策略
资源路线
自动解锁
动态调参
异常恢复
高效路径
并行调度
后期成就路线

在这个阶段，代码不再只是某个农场脚本，而是一套完整系统。

设计价值

排行榜阶段相当于把前面所有系统重新压缩成一个优化问题：

最短时间内完成最大进度

这会自然催生：

阶段切换
状态机
成本预测
资源瓶颈分析
并行任务调度
特殊模式脚本

这是游戏最终的系统深度。

十二、整体体验周期总结

《编程农场》的体验周期可以概括为：

1. 手动操作
2. 基础循环
3. 资源阈值轮作
4. 生产链维护
5. 特殊作物策略
6. 资源转换链
7. 寻路问题
8. 自动解锁系统
9. 测量 / 最值选择
10. 多无人机并行
11. 排序 / 图依赖 / 特殊移动规则
12. 完全自动化与排行榜

每个阶段都不是简单加数值，而是增加新的编程概念：

循环
条件判断
状态机
路径规划
字典
自动购买
最值搜索
并发
排序
图依赖
自碰撞路径
完整自动化

这就是它最核心的内容设计方式。

十三、数值设计原则总结

1. 用资源链制造隐性瓶颈

高级资源不要直接生产，而是通过低级资源转换：

Pumpkin 依赖 Carrot
Weird Substance 依赖 Fertilizer
Gold 依赖 Maze

Power 依赖 Sunflower
Bone 依赖 Dinosaur

这样玩家不能只优化单点，而要维护系统。

2. 用面积平方放大规模感

农场扩张不是线性成长，而是面积成长：

$N \times N$

这让扩张具备强烈吸引力。

但扩张也带来巡田成本、水消耗、Power 消耗、无人机调度压力，因此不是无脑越大越好。

3. 用特殊作物打破通用脚本

每个高级作物都应该破坏玩家旧代码假设：

树：不能密集种
南瓜：不能单格成熟就收
向日葵：不能见熟就收，要收最大
仙人掌：不能乱收，要排序
恐龙：不能普通导航，要避开尾巴

这让内容持续产生“重构需求”。

4. 用 API 解锁作为内容奖励

游戏的奖励不是只有资源，也包括新的表达能力：

```
import  
dictionary  
get_cost  
unlock  
measure  
swap  
simulate  
random  
spawn_drone
```

这些 API 会改变玩家写代码的方式，因此比普通数值奖励更有长期吸引力。

5. 用动态调参替代固定阈值

随着农场变大，固定目标会失效。

更好的自动化目标是：

目标库存 = 地块数 × 每格储备
升级需求 = get_cost(下一个目标)
生产模式 = 当前缺口资源

后期真正的策略不是写死数值，而是写一个会根据科技树和成本自适应的系统。

6. 对流动资源使用节流，而不是库存追逐

Power、Gold 在某些阶段容易形成循环。

尤其 Power 会被移动消耗，如果把它当普通库存追，会导致：

刷 Power
→ 移动消耗
→ 继续刷 Power

所以流动资源需要节流：

只在升级明确需要时生产
每次生产后冷却
收获后立刻尝试 unlock
不要长期占用主循环

十四、对游戏设计的启发

《编程农场》的厉害之处在于：
它不是把编程包装成游戏，而是把游戏内容设计成一系列越来越复杂的编程问题。

好的阶段设计不是：

新增一种作物
新增一种资源
新增一个升级

而是：

新增一种会让旧代码失效的新规则

例如：

树让密集种植失效
南瓜让单格收割失效

向日葵让无脑收割失效
仙人掌让无序收割失效
恐龙让普通移动失效
巨大农场让单无人机遍历失效

这使得玩家每次进入新阶段时，都不是简单重复旧脚本，而是必须重构思路。

这就是它内容填充的核心：

用机制变化制造代码重构
用数值压力推动玩家升级架构
用 API 解锁扩展玩家表达能力
用资源链维持长期目标

因此，《编程农场》的体验曲线可以看作：

从写脚本
到写系统
到写自动化经济
到写算法集合
到写完整生产框架

这也是它区别于普通自动化游戏的地方。